Tutorial 1: Read a text file

Objective

This lab aims to introduce you to the concept of events. By the end of this session, you should have

- Obtained a mental image of events
- Understood the concept of event rate
- Gain insight on the computation time of events
- Begin to understand the concept of event representations

Programming the code yourself is strongly recommended, to build the understanding of the data.

Exercise

In this exercise you will read and manipulate events from a .txt file. Download the file events_chunk.txt, you will be working with it for this exercise.

Environment setup

To solve the exercises from this lesson we are suggesting the following libraries:

- OpenCV (python-opencv, cv2)
- NumPy
- Matplotlib

Instructions

(a) Read the file events_chunk.txt. At each line in the file, a sequence of 4 values are present. `timestamp, x, y, polarity`. Create 4 lists. Now for each line of the text file, append these 4 values to their respective lists. At the end, you should have 4 identically shaped lists.

(b) Create a plot showing the rate of events over time. Divide the time span covered by the events (Hint: this is the first and last timestamp of your data) into 10 bins, and count the number of events that occur within each time bin. Plot the number of events, or event rate as a function of time. Make sure to label both axes and include appropriate units. What do you observe? Try to increase the number of bins. How does this change your understanding of the data?

(c) Create an array of zeros of size (180,240) (Hint: you can use the command np.zeros()). Now for each of the first 5000 events in your lists, add a 1 to its respective x, y location in the array. Disregard the polarity for this exercise. Display the results, what do you see? (Note: if it is all black something is not right.) Let us call this an event frame.

(d) Create a loop, that parses all events, and with a step of 1000 events (eg. index 0-4999, then 1000-5999, 2000 - 6999), it creates an event frame with the following 5000 events. Display the results as a video. For this you can display each frame with:

```
cv2.imshow("Event Stream", frame)
cv2.waitKey(1)
```

and use `cv2.destroyAllWindows()` at the end. If the display flickers too fast or looks odd, try adjusting the delay in `cv2.waitKey()`

What do you see? If it looks strange, how would you fix it?

(e) Redo parts c and d. This time, create a colored image. Add positive polarity events in the red channel and negative polarity events in the blue channel.

(f) Resize the results to improve visibility.

(g) Create a new list. For each frame in the video, note the time span of events that the frame encompasses (last-timestamp - first-timestamp) into this list. Plot the values in the list. Compare this plot to the plot obtained in b. What do you see?

Submission

NO SUBMISSION REQUIRED FOR THIS TUTORIAL.